PROJECT DOCUMENTATION

Voice-Based Concept Understanding Analyzer

(VBCUA)

An AI-Powered System for Evaluating Spoken Conceptual Understanding Using Whisper and Sentence Transformers

---

Submitted in Partial Fulfillment of the Requirements for the SmartBridge AI & Machine Learning Internship Program

Developed By

Team ID: _____

Team Members:

- Venkata Krishna Vamsi Vemula
- Nikhilkumar Ghanta
- Balineni Lokesh
- Bale Lokesh Manikanta
- Jaswanth Sai Tangudu

Guided By

SmartBridge

Submitted On

14th July 2026

_____

. Table of Contents

1. Project Description

The Voice-Based Concept Understanding Analyzer (VBCUA) is an AI-powered educational application that evaluates a student's conceptual understanding directly from a spoken explanation. The system converts a student's recorded audio answer into text using OpenAI Whisper, compares the transcript against a stored reference answer using Sentence Transformer embeddings, computes a semantic similarity score, and produces a grade with descriptive feedback. All results, including the transcript, similarity score, grade, and feedback, are compiled into a downloadable PDF report.

The application is delivered as an interactive Streamlit web interface, allowing a student or evaluator to upload an audio file (WAV, MP3, or M4A) and immediately view the transcription, similarity metrics, and generated assessment without any manual intervention.

VBCUA is organized as a modular Python package. Each processing stage — transcription, semantic analysis, scoring, and report generation — is implemented as an independent module under app/modules, while presentation logic is isolated under app/ui. This separation keeps the pipeline easy to test, extend, and maintain.

2. Application Scenarios / Use Cases

Based on the implemented pipeline, VBCUA is suited to scenarios where a spoken response needs to be objectively compared against a known-correct explanation:

- Formative self-assessment: A student records an explanation of a concept (e.g., "Inheritance in OOP") and instantly receives a similarity score, grade, and feedback.
- Oral viva / assignment support: An instructor uses the tool as a first-pass, automated check of a student's verbal understanding before a manual review.
- Flipped-classroom practice: Students verbally rehearse answers to conceptual questions and use the generated PDF report to track improvement over repeated attempts.
- Corporate or vocational training checks: Trainees explain a procedure or concept aloud, and the tool benchmarks the explanation against an approved reference answer.
- Accessibility-friendly assessment: Learners who find typed answers difficult can be assessed through speech instead of writing.

3. Technical Architecture Overview

VBCUA follows a layered, modular architecture. The Streamlit UI layer handles file upload and result presentation, the processing layer performs transcription and semantic analysis, and a persistence layer stores uploaded audio, reference answers, and generated reports on the local filesystem.

Architecture Diagram

Presentation Layer (app/ui)
upload_section.py · results_panel.py · report_actions.py · sidebar.py — built with Streamlit widgets

Application Entry Point (app/main.py)
Orchestrates page config, session state, and calls into processing modules

Processing Layer (app/modules)
transcriber.py (Whisper) · semantic_analyzer.py (Sentence Transformers) · concept_scorer.py · report_generator.py (ReportLab)

Utility Layer (app/utils)
file_io.py — reference answer loading; config.py, paths.py, validators.py, logger.py (scaffolded for future use)

Data / Storage Layer
data/audio/ (uploaded audio) · data/reference_answers/ (ground-truth text) · reports/ (generated PDF reports)

Component Responsibilities

Component	Responsibility
transcriber.py	Loads and caches the Whisper "base" model; transcribes an audio file to plain text.
semantic_analyzer.py	Loads and caches the all-MiniLM-L6-v2 Sentence Transformer; computes cosine similarity.
concept_scorer.py	Maps a similarity percentage to a grade (Excellent / Good / Fair / Needs Improvement) with feedback.
report_generator.py	Builds a PDF report (title, transcript, score, grade, feedback, timestamp) using ReportLab.

Component	Responsibility
main.py	Coordinates upload, transcription, scoring, and report generation; manages Streamlit session state.

4. Prerequisites

Software Requirements

- Python 3.12
- pip package manager
- Virtual environment tool (venv)
- FFmpeg (required by Whisper for audio decoding)
- Git (for cloning the repository)

Hardware Recommendations

- Minimum 8 GB RAM (Whisper and Sentence Transformer models are loaded into memory)
- CPU is sufficient for the "base" Whisper model; a GPU is optional and improves transcription speed
- At least 2 GB free disk space for model downloads and dependencies

Key Python Dependencies

Package	Version	Purpose
streamlit	1.58.0	Web application framework
openai-whisper	20250625	Speech-to-text transcription
sentence-transformers	5.6.0	Semantic embeddings and similarity
librosa	0.11.0	Audio loading and analysis utilities
reportlab	5.0.0	PDF report generation
torch	2.12.1	Deep learning backend for Whisper and Sentence Transformers

5. Prior Knowledge Required

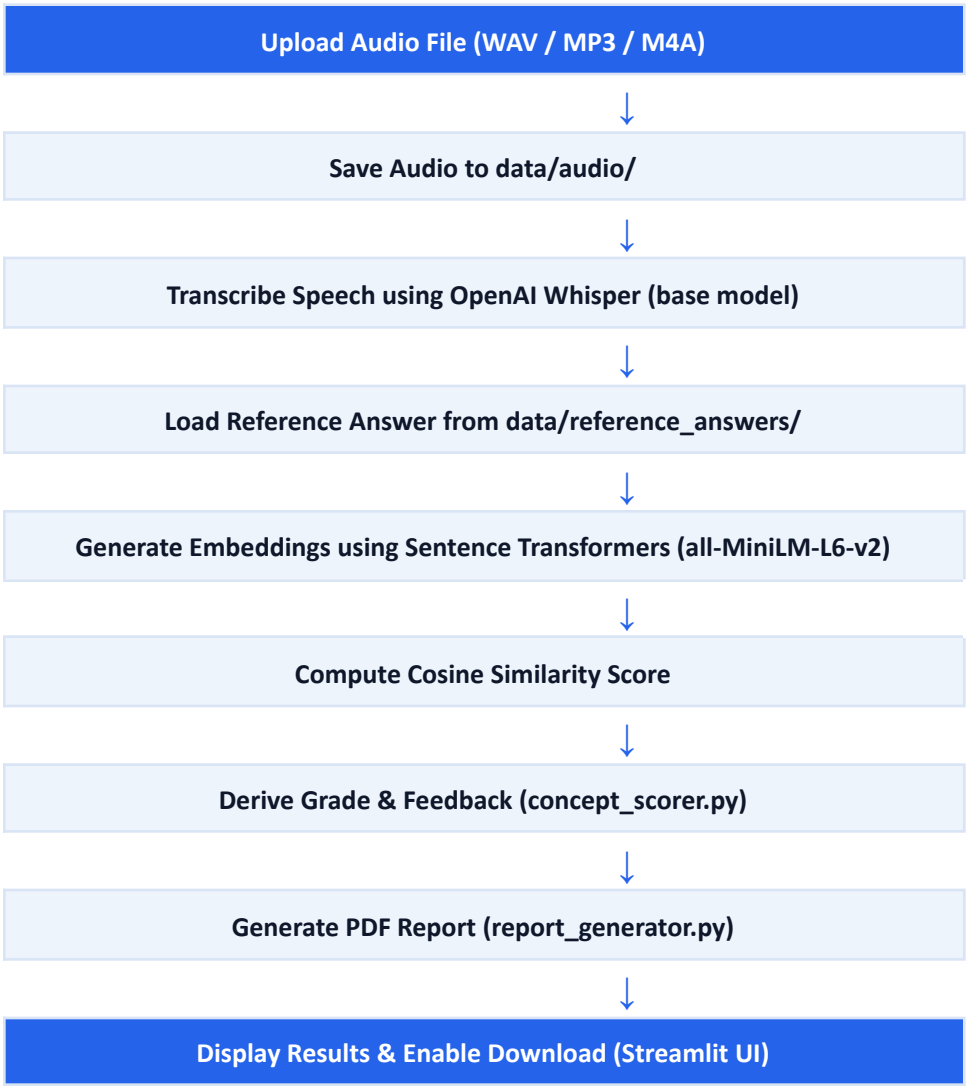
- Core Python programming (functions, modules, exception handling)
- Basic understanding of the Streamlit framework for building web UIs
- Fundamentals of Natural Language Processing, particularly text embeddings and cosine similarity
- Basic familiarity with speech-to-text concepts and pretrained model usage
- Basic file handling and virtual environment management in Python
- Elementary understanding of PDF generation libraries (helpful, not mandatory)

6. Project Objectives

#	Objective	Status
1	Speech-to-text transcription of student audio responses	Implemented
2	Semantic similarity analysis between transcript and reference answer	Implemented
3	Automated concept understanding scoring	Implemented
4	Downloadable PDF assessment reports	Implemented
5	Interactive, user-friendly Streamlit interface	Implemented

7. System Workflow

The end-to-end workflow implemented in app/main.py proceeds as follows:



7. Milestone 1: Requirement Analysis & System Design

This milestone focused on defining the problem statement — assessing conceptual understanding from spoken answers — and designing a modular pipeline capable of transcription, semantic comparison, scoring, and reporting. The package layout (app/modules, app/ui, app/utils) and the data flow between components were finalized during this phase.

Key Deliverables

- Problem statement and scope definition
- Selection of technology stack: Whisper, Sentence Transformers, Streamlit, ReportLab
- Modular package structure design (modules / ui / utils separation)
- Definition of core data contracts: transcript (str), similarity_score (float 0.0-1.0), score_data (dict with score/grade/feedback)

8. Milestone 2: Environment Setup & Initial Configuration

The development environment was established with a Python virtual environment and a pinned requirements.txt covering all AI, audio, UI, and reporting dependencies. Project scaffolding files (config.py, paths.py, logger.py, validators.py) were created to support future configuration needs, and Streamlit theming was configured via .streamlit/config.toml.

Key Deliverables

- Virtual environment (.venv) creation and dependency installation via requirements.txt
- Streamlit theme configuration (primary color, backgrounds, upload size limit) in .streamlit/config.toml
- Project directory scaffolding: data/audio, data/reference_answers, reports, tests
- .gitignore configured to exclude virtual environments, audio uploads, generated reports, and secrets

9. Milestone 3: Core System Development

The core AI processing modules were implemented: audio transcription with Whisper, semantic similarity computation with Sentence Transformers, and rule-based concept scoring. Each module includes input validation and dedicated exception types (TranscriptionError, SemanticAnalysisError) for predictable error handling.

Key Deliverables

- transcriber.py: cached Whisper "base" model loading and transcribe_audio() function
- semantic_analyzer.py: cached SentenceTransformer (all-MiniLM-L6-v2) loading and calculate_similarity() using cosine similarity
- concept_scorer.py: score_concept_understanding() mapping similarity percentage to grade tiers (Excellent ≥ 90%, Good ≥ 75%, Fair ≥ 60%, Needs Improvement < 60%)
- file_io.py: load_reference_answer() for reading ground-truth text files

10. Milestone 4: Integration & Optimization

The individual modules were integrated into a single Streamlit application (main.py) with session-state caching so that repeated interactions do not re-run transcription unnecessarily. The UI layer (upload_section.py, results_panel.py, report_actions.py, sidebar.py) was built to present transcripts, scores, grades, and feedback in a clear dashboard layout, and report_generator.py was integrated to automatically produce a PDF after every successful analysis.

Key Deliverables

- Session-state based caching keyed by file name and size to avoid redundant re-transcription
- Integrated results dashboard: similarity score metric, color-coded grade badge, transcript viewer, feedback panel
- Automatic PDF report generation triggered immediately after scoring
- Custom corporate theme (colors, metric cards, upload box styling) applied via injected CSS

11. Milestone 5: Testing & Validation

Validation focused on boundary and error-handling behavior across the pipeline: invalid similarity scores, missing or empty reference answers, missing audio files, and malformed score data are all handled with explicit exceptions and user-facing warnings rather than uncaught crashes. A tests/ package was scaffolded for unit test coverage.

Key Deliverables

- Input validation in concept_scorer.py and report_generator.py (score range 0.0–1.0, non-empty transcript/grade/feedback)
- Graceful error handling in main.py for FileNotFoundError, TranscriptionError, SemanticAnalysisError, and PDF generation failures
- Manual end-to-end validation using the sample reference answer (data/reference_answers/sample_answer.txt) covering the "Inheritance" concept
- tests/ package scaffolded for future automated unit and integration tests

12. Deployment

The application currently runs as a local Streamlit server, launched with a single command after dependency installation:

```
streamlit run app/main.py
```

Streamlit's built-in development server hosts the interface on http://localhost:8501 by default. Application-wide UI settings, such as the primary color and maximum upload size (200 MB), are configured through .streamlit/config.toml, and secrets are expected in .streamlit/secrets.toml, which is excluded from version control.

Current Deployment Model

- Local, single-user deployment via the Streamlit development server
- Uploaded audio and generated reports are stored on the local filesystem (data/audio/, reports/)

Future Deployment Enhancements

- Cloud deployment (e.g., Streamlit Community Cloud or a containerized service) — not yet implemented
- Authentication and multi-user session isolation — not yet implemented

13. Project Structure

The repository follows the modular layout below, mirroring the separation between processing modules, UI components, and utility helpers:

```
Voice-Based-Concept-Analyzer/
├── app/
│   ├── modules/
│   │   ├── transcriber.py
│   │   ├── semantic_analyzer.py
│   │   ├── concept_scorer.py
│   │   └── report_generator.py
│   ├── ui/
│   │   ├── sidebar.py
│   │   ├── upload_section.py
│   │   ├── results_panel.py
│   │   └── report_actions.py
│   ├── utils/
│   │   ├── file_io.py, paths.py, validators.py, logger.py
│   ├── config.py
│   └── main.py
├── data/
│   ├── audio/
│   └── reference_answers/sample_answer.txt
├── reports/
├── tests/
├── .streamlit/config.toml
├── requirements.txt
```

```
├── .gitignore
└── README.md
```

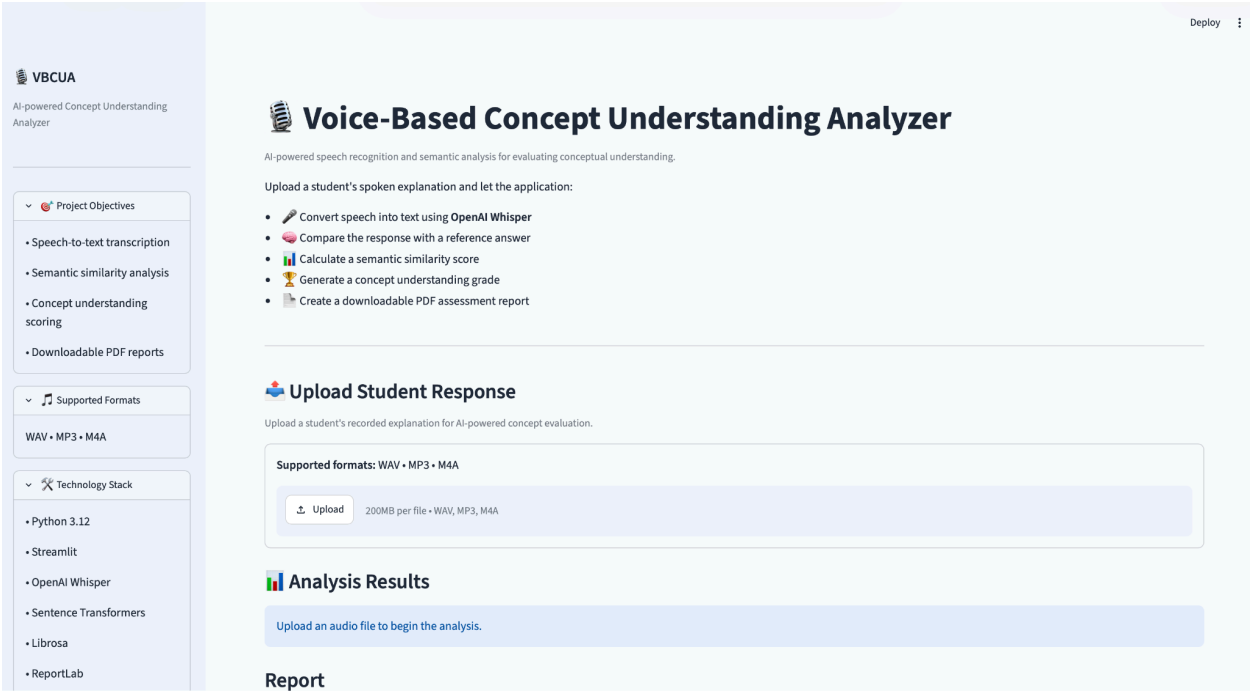
14. Results

Using the sample reference answer on Inheritance in Object-Oriented Programming, a matching spoken explanation produces the following representative output from the implemented pipeline:

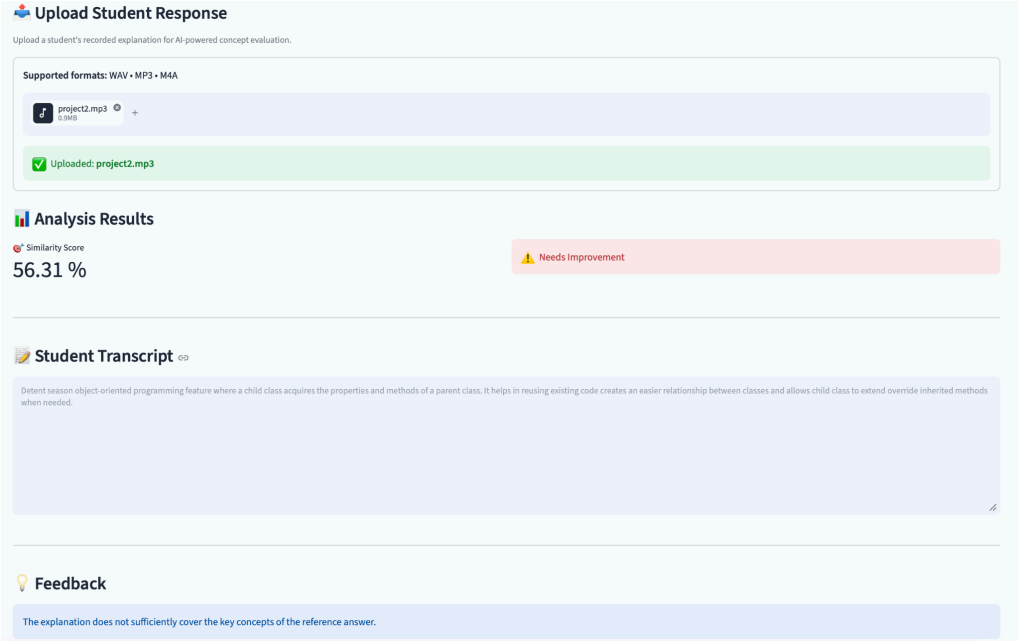
Metric	Value
Concept Understanding Score	97.77 %
Grade	Excellent
Feedback	The explanation closely matches the reference answer and demonstrates excellent conceptual understanding.

The results panel displays this score alongside a color-coded grade badge (green for Excellent, blue for Good, amber for Fair, red for Needs Improvement), the full transcript, and the feedback text. A matching PDF report is generated automatically and made available through the download button.

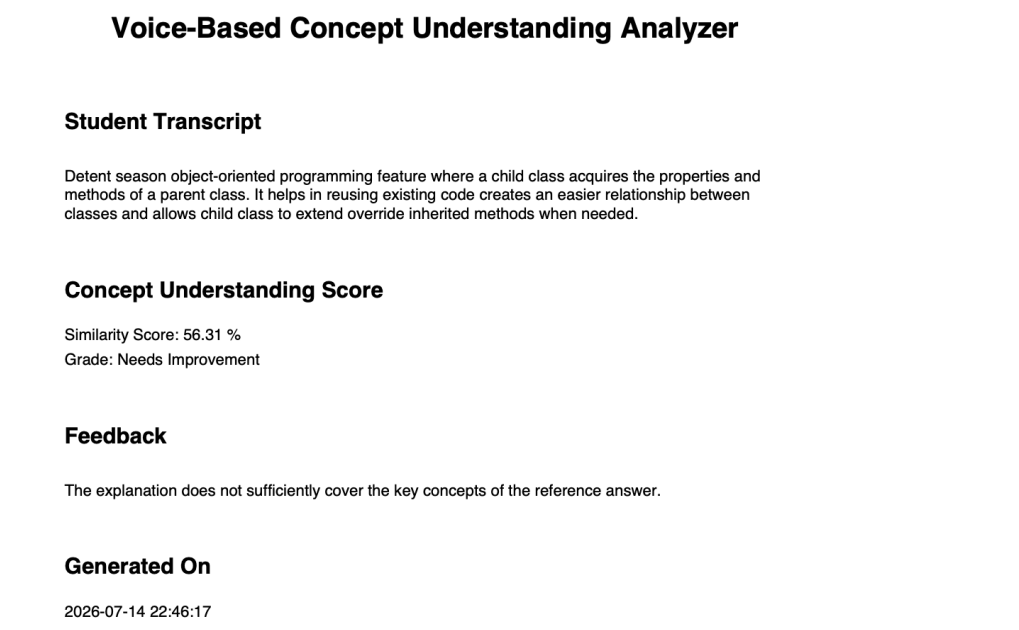
Home Page ScreenShot



Result Generated ScreenShot



Result PDF ScreenShot



Voice-Based Concept Understanding Analyzer

Student Transcript

Detent season object-oriented programming feature where a child class acquires the properties and methods of a parent class. It helps in reusing existing code creates an easier relationship between classes and allows child class to extend override inherited methods when needed.

Concept Understanding Score

Similarity Score: 56.31 %
Grade: Needs Improvement

Feedback

The explanation does not sufficiently cover the key concepts of the reference answer.

Generated On

2026-07-14 22:46:17

15. Advantages & Limitations

Advantages

- Fully automated pipeline from raw audio to a graded, feedback-rich PDF report
- Modular design allows individual components (transcription, scoring, reporting) to be replaced or upgraded independently
- Semantic comparison (rather than keyword matching) tolerates paraphrasing and varied sentence structure
- Clear, explicit error handling with user-facing messages instead of silent failures
- Lightweight, local-first architecture with no external API dependency for inference

Limitations

- Supports a single, fixed reference answer per evaluation; no multi-reference-answer comparison yet
- No real-time microphone recording; requires a pre-recorded audio file
- Single-language support (models are used in their default configuration)
- No persistent database — results and reports exist only for the current session and local filesystem
- No authentication or multi-user isolation, limiting it to single-user local use today

16. Future Enhancements

- Real-time microphone recording directly within the Streamlit interface
- Support for multiple reference answers per concept
- Multi-language transcription and semantic comparison
- Student performance dashboard with historical score tracking
- Database integration for persistent storage of transcripts and results
- Cloud deployment for multi-user access
- Authentication and user management

17. Conclusion

VBCUA successfully demonstrates an end-to-end, AI-powered pipeline that transforms a spoken explanation into an objective, semantically-grounded assessment. By combining OpenAI Whisper for transcription, Sentence Transformers for semantic similarity, a rule-based scoring module, and automated PDF reporting, the project delivers a functional tool for evaluating conceptual understanding without manual grading of transcripts.

The modular architecture — with clearly separated processing, UI, and utility layers — provides a solid foundation for the future enhancements identified above, including multi-reference support, real-time recording, and cloud deployment, without requiring a redesign of the existing pipeline.